Homework 2

Points: 2

Deadline: 03.11.2026 01:00 a.m. (System time in Blackboard)

Our system is in an undesired state: Many transactions are aborted by the DBMS, because it detects some foolish “deadlocks” it says. We have to quickly find the problem and engineer a working solution.

Your mission, should you choose to accept it, is to identify the culprits causing the transactions to abort and neutralize them.

In the Java file attached, you find a tiny class (Blocking) creating a table and various tasks named Blocker. Each Blocker-Task issues multiple update statements within one transaction. Recapitulate, this means that the corresponding transaction of a task therefore needs to issue *write locks* to the table and holds them until it commits (or gets aborted by the DBMS). The tasks run concurrently, such that the transactions can block each other, which may lead to deadlocks.

Your missions are now the following three:



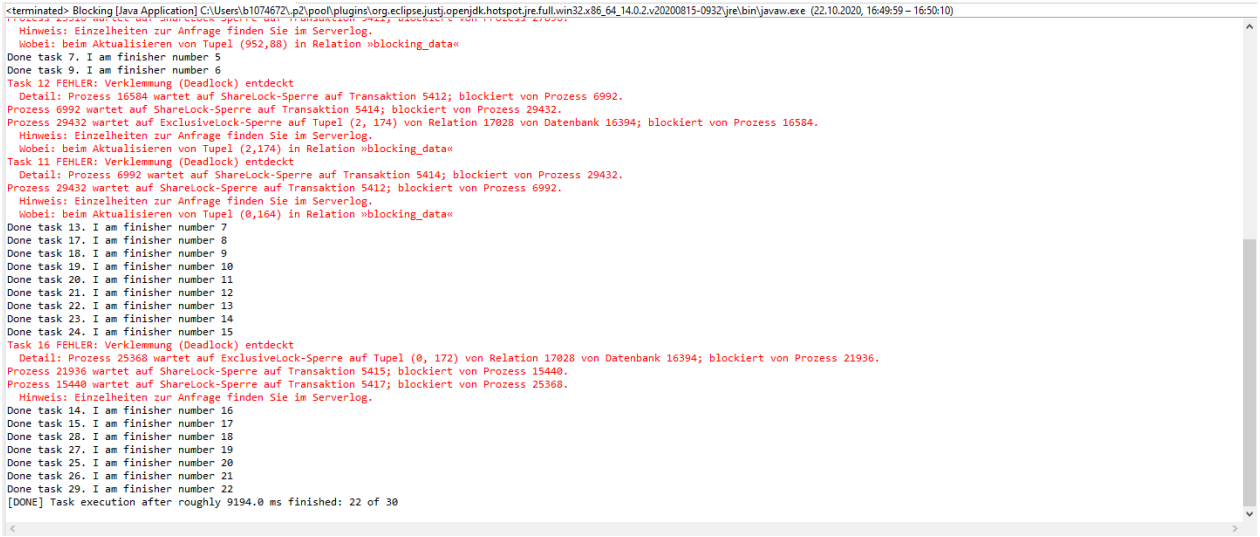
1) Preliminaries

- Create a GitHub (or GitLab) repo using the template project at link.
    - Use your private GitHub account or create a new one.
    - Make the project public.
- Install PostgresSQL database system on your computer.
- Install Java and some IDE of your choice. We recommend IntelliJ.

Hint: In case you do not like Java. You may program an equivalent solution in a programming language of your choice. We use the same set up also for the next task. In the end, there must be some ThreadPool spawning a lot of these Blocker tasks, such that the database system detects deadlocks. In case you want to come up with your own implementation, contact me.

2) Make the Java Program run (0.5 points)

Prepare a live demo for your talk indicating that the program indeed produces deadlocks. Verify that the error messages you receive indicate that the system uses row-level locks (i.e., no table or page locks). Also, report on issues or problems you faced when running the program. To give you an example how it should look like, my console output looks like this:



In case you face issues producing deadlocks check the following:

- In case the compiler complains about non-ascii characters: Remove them.
- You may change the package structure.
- I used PostgreSQL 16, with JDBC driver 4.2 in version 42.2.18 downloaded from: https://jdbc.postgresql.org/download.html
- Isolation level should be read committed (Line 153)
- The missing ConnectionFactory class simply builds the connection String and then issues: `conn = DriverManager.getConnection(url, props);` Simply replace this with your own connection settings. You do not have to upload this class into your git repository.

3) Restart the aborted Transactions (0.5 points)

In the initial solution, you are supposed to modify the program, such that aborted transactions are restarted (from the client side) until they commit. To this end, submit a new Java File named "RestartBlocking.java" with your modifications. Note, I will test your submission with varying number of tasks, queries per task, and number of concurrent threads.

For the talk, also include a small demo or screenshots, explain the general idea, and report on most important issues you faced upon solving this task.

4) Avoid starvation (1 point)

You quickly see that the prior solution means that the more transactions we have, the more get aborted and restarted, and maybe aborted again. It may even happen that some threads always get aborted, so they never finish. We call this a life lock or starvation.

1. Figure out whether this issue occurs in your setting. Note, this may vary from machine to machine and may even be not occur at all. Nevertheless, if you do not observe a life lock immediately, try to provoke one by altering the level parallelism, transaction number etc.
2. Develop a concept to solve this problem and implement it in another File named "NoLiveLocks.java". In the talk, explain how you tackle this problem. In addition, include a small Live Demo, and discuss the most severe challenges. Compare the runtime to the restart solution above.

Note: Do not try to squeeze out the last nanosecond of performance. This is the next task.

Useful hints and reading

`pg_stat_activity` view shows information related to the current activity of processes.
https://www.postgresql.org/docs/current/monitoring-stats.html

`pg_blocking_pids(int)` lists process `pid`s that are blocking specified process `pid` from acquiring a lock.
https://www.postgresql.org/docs/current/functions-info.html

`deadlock_timeout (integer)` details server options for lock management.
https://www.postgresql.org/docs/current/runtime-config-locks.html

5) Deliveries until Deadline

- In Git: Source code for the solutions. We grade the last commit in the git classroom before submission deadline. Include the LINK to your repo in the Blackboardsubmission
- In Blackboard: The slides for the talk.
    - Introduce your team and task distribution
    - Include what were the major challenges you faced upon working on the task and what worked out well.
    - Was there something new for (at least one of) you?
    - Shortly explain the concept and show that it works
    - Include information about your time management.
    - The talk should be 8 minutes. I'll abort the talk if it takes too long.
    - Include an estimation of the percentages the group members contributed to the solution